# Oversættelsesregler

Til "Stack machine" codelab

I det følgende er code(*e*) oversættelsen af udtryk/expression *e*, mens code(*s*) er oversættelsen af kommando/statement *s* og code(*d*) er oversættelsen af erklæring/declaration *d*.

## Literaler

| | | |
|---|---|---|
| *n* | **`push`** *n* | hvor *n* er er en heltalsliteral som fx `-4`, `0`, `3` |
| *b* | **`push`** *b* | hvor *b* er er en boolesk literal, `true` eller `false` |

## Aritmetiske udtryk

| | |
|---|---|
| $e_1$ `+` $e_2$ | code($e_1$)<br>code($e_2$)<br>**`add`** |
| $e_1$ `-` $e_2$ | code($e_1$)<br>code($e_2$)<br>**`sub`** |
| $e_1$ `*` $e_2$ | code($e_1$)<br>code($e_2$)<br>**`mul`** |
| $e_1$ `/` $e_2$ | code($e_1$)<br>code($e_2$)<br>**`div`** |
| $e_1$ `%` $e_2$ | code($e_1$)<br>code($e_2$)<br>**`mod`** |
| `-`*e* | code(*e*)<br>**`neg`** |
| $e^2$ | code(*e*)<br>**`dup`**<br>**`mul`** |

# Relationelle udtryk

| | |
|---|---|
| $e_1$ = $e_2$ | code($e_1$)<br>code($e_2$)<br>**eq** |
| $e_1$ ≠ $e_2$ | code($e_1$)<br>code($e_2$)<br>**ne** |
| $e_1$ ≤ $e_2$ | code($e_1$)<br>code($e_2$)<br>**le** |
| $e_1$ ≥ $e_2$ | code($e_1$)<br>code($e_2$)<br>**ge** |
| $e_1$ < $e_2$ | code($e_1$)<br>code($e_2$)<br>**lt** |
| $e_1$ > $e_2$ | code($e_1$)<br>code($e_2$)<br>**gt** |

# Logiske udtryk

| | |
|---|---|
| $e_1$ **and** $e_2$ | code($e_1$)<br>**jmpf** else<br>code($e_2$)<br>**jump** done<br>else:<br>**push false**<br>done: |
| $e_1$ **or** $e_2$ | code($e_1$)<br>**jmpf** else<br>**push true**<br>**jump** done<br>else:<br>code($e_2$)<br>done: |
| **not** $e$ | code($e$)<br>**not** |

# Betingede udtryk

| | |
|---|---|
| $e_1$ **then** $e_2$ **else** $e_3$ | code($e_1$)<br>**jmpf** else<br>code($e_2$)<br>**jump** done<br>else:<br>code($e_3$)<br>done: |

# Sideeffekter

| | |
|---|---|
| **read** | **read** |
| **write** $e$ | code($e$)<br>**wrt** |
| **eval** $e$ | code($e$)<br>**pop** |

# Globale variabler

I oversættelsen af programmer med globale variabler gør vi os den implicitte antagelse, at værdierne af variablerne $x_0,\ldots,x_{n-1}$ befinder sig i de $n$ nederste celler på stakken.

| | | |
|---|---|---|
| **global** $x_n$ = $e$ | code($e$) | Herefter er $x_n$ også på stakken. |
| $x_i$ | **load** $i$ | |
| **set** $x_i$ = $e$ | code($e$)<br>**save** $i$ | |

# Lokale variabler

I oversættelsen af programmer med lokale variabler gør vi os den implicitte antagelse, at værdierne af variablerne $x_0,\ldots,x_{n-1}$ befinder sig i de $n$ nederste celler på stakken – eller i de $n$ nederste celler af nuværende *frame*, hvis et procedurekald er aktivt.

| | | |
|---|---|---|
| **let** $x_n$ = $e$ **in**<br>$s$<br>**end let** | code($e$)<br>code($s$)<br>**pop** | Her er $x_n$ også på stakken |
| $x_i$ | **load** $i$ | |
| **set** $x_i$ = $e$ | code($e$)<br>**save** $i$ | |

## Sekvens

| | |
|---|---|
| $s_1$ | code($s_1$) |
| $s_2$ | code($s_2$) |
| ⋮ | ⋮ |
| $s_k$ | code($s_k$) |

## Selektion

| | |
|---|---|
| **if** $e$ **then** | code($e$) |
| $s_1$ | **jmpf** else |
| **else** | code($s_1$) |
| $s_2$ | **jump** done |
| **end if** | else: |
| | code($s_2$) |
| | done: |

| | |
|---|---|
| **if** $e$ **then** | code($e$) |
| $s$ | **jmpf** done |
| **end if** | code($s$) |
| | done: |

## Iteration

| | |
|---|---|
| **while** $e$ **do** | loop: |
| $s$ | code($e$) |
| **end while** | **jmpf** done |
| | code($s$) |
| | **jump** loop |
| | done: |

| | |
|---|---|
| **for** ( **let** $x_n$ = $e_1$ ; $e_2$ ; $s_3$ ) | code($e_1$) |
| $s$ | loop: |
| **end for** | code($e_2$) |
| | **jmpf** done |
| | code($s$) |
| | code($s_3$) |
| | **jump** loop |
| | done: |
| | **pop** |

Her er $x_n$ også på stakken

## Funktionskald

| | |
|---|---|
| $f(e_1, e_2, \ldots, e_n)$ | **push** $f$<br>code($e_1$)<br>code($e_2$)<br>⋮<br>code($e_n$)<br>**call** $n$ |

## Funktions-erklæring

| | |
|---|---|
| **func** $f(x_0, x_1, \ldots, x_{n-1})$ **is**<br>  $s$<br>**end func** | $f$:<br>code($s$) |
| **return** $e$ | code($e$)<br>**retv** |

## Procedurekald

| | |
|---|---|
| $p(e_1, e_2, \ldots, e_n)$ | **push** $p$<br>code($e_1$)<br>code($e_2$)<br>⋮<br>code($e_n$)<br>**call** $n$ |

## Procedure-erklæring

| | |
|---|---|
| **proc** $p(x_0, x_1, \ldots, x_{n-1})$ **is**<br>  $s$<br>**end proc** | $p$:<br>code($s$)<br>**ret** |
| **return** | **ret** |

## Program

| | |
|---|---|
| $s$ | code($s$)<br>**halt** |
| $d_1$ | code($d_1$) |
| $d_2$ | code($d_2$) |
| ⋮ | ⋮ |
| $d_k$ | code($d_k$) |